Geo-Distributed NoSQL: Core Idea

The Challenge

Data scattered across regions creates latency spikes, regional failures, and network partitions

The Solution

Store replicas of data in multiple regions for speed, availability, and resilience

Low Latency

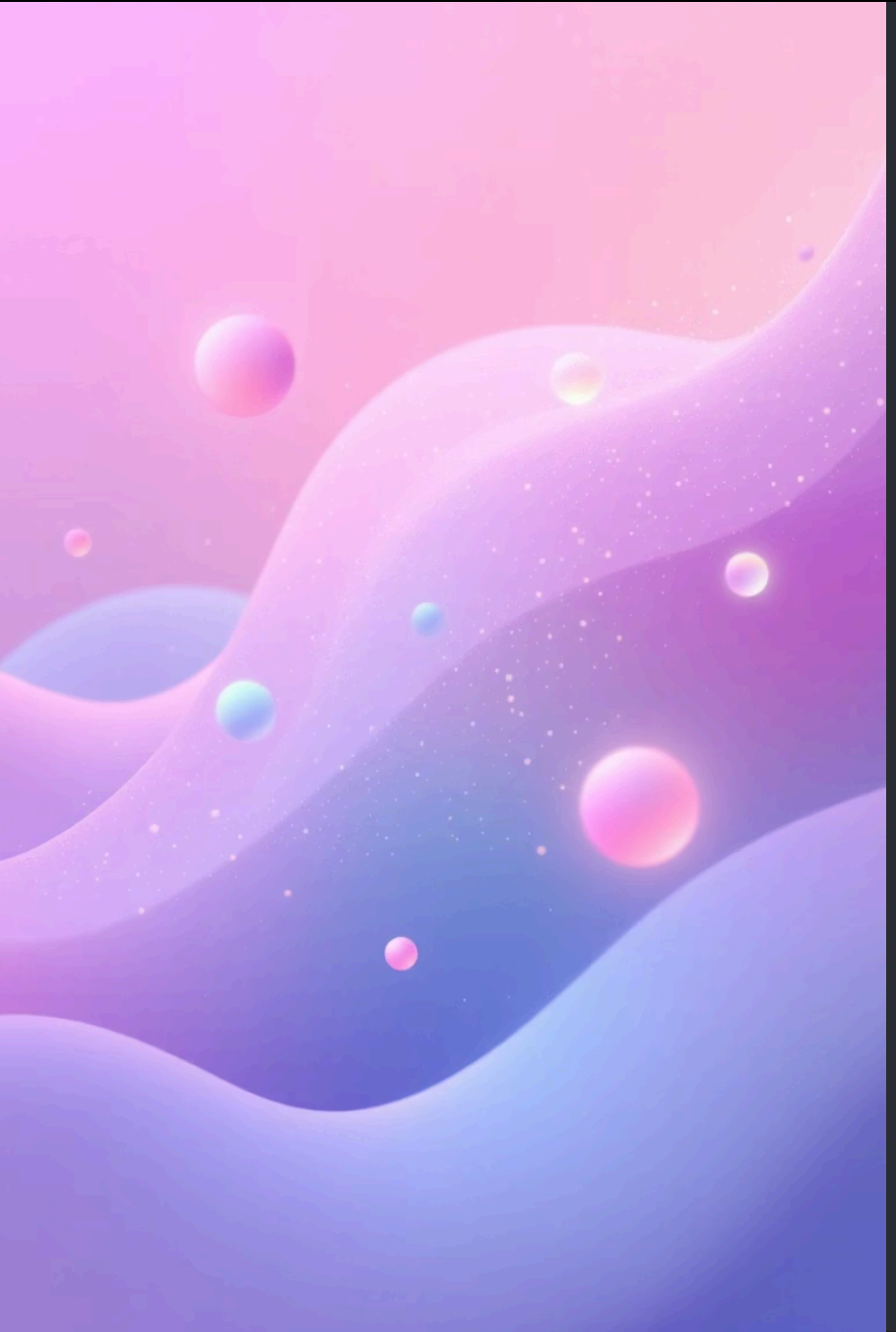
Users read from nearest region

High Availability

Region failure ≠ system outage

Fault Tolerance

Hardware & network resilience



Replication Strategies

Synchronous

All replicas ACK before response

Strong consistency, higher latency

Asynchronous

Primary ACKs first, others sync later

Fast writes, eventual consistency

Semi-Synchronous

ACK from subset of replicas

Practical balance between both

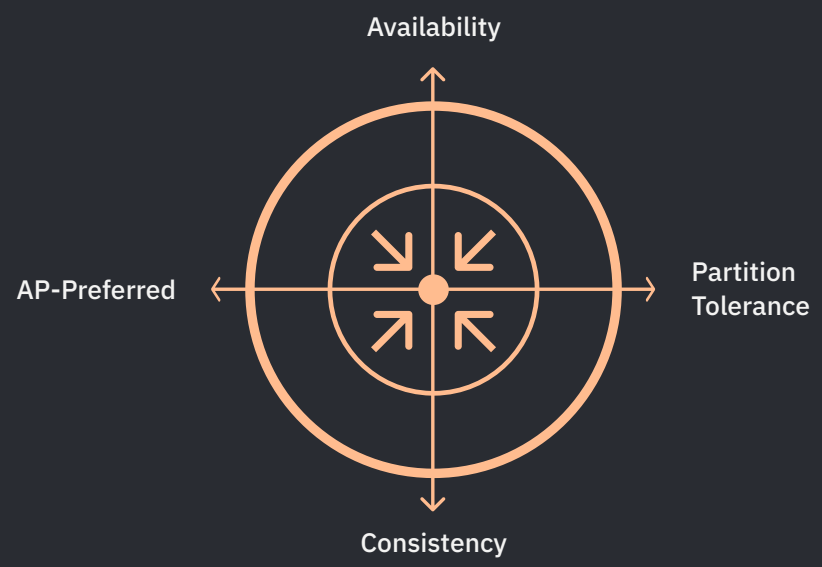
Replica Management

- Creation & placement
- Update propagation
- Health monitoring
- Regional compliance

Placement Strategy

Per-continent replicas with primary-secondary architecture ensuring data residency and compliance requirements

Consistency Models & Protocols



NoSQL systems prioritize **Availability** and **Partition Tolerance**, making consistency tunable

Strong Consistency
Always return latest value

Eventual Consistency
Replicas converge over time

Causal Consistency
Related operations in order

Read-Your-Writes
Users see their own updates

Common Protocols

Quorum (N, R, W)
R + W > N ensures overlap

Leader–Follower
Single write coordinator

Multi-Leader
Conflict resolution needed

Consensus
Raft / Paxos algorithms

Docker-Based Quorum Implementation

Replica Architecture

3 Docker containers simulate geo-distributed regions:

- Mumbai → Port 5001
- Virginia → Port 5002
- Frankfurt → Port 5003

Container isolation mimics data-center separation

Quorum Config

A **quorum** is the **minimum number of replicas that must agree** for a read or write operation to be considered valid in a distributed system.

N = 3, W = 2, R = 2

Guarantees read-write overlap for consistency

Write Flow

Forward to all replicas, ACK after **W=2** confirmations

Read Flow

Query **R=2** replicas, resolve conflicts, return latest

Result

High availability with eventual consistency guarantee